ChromSimPipe Code Guide

End-to-end walkthrough of the cohesin loop-extrusion simulation codebase

2026

Contents

CODE_GUIDE: What Every File in This Repo Actually Does 1

- Quick-Start and Codebase Walkthrough . . . 2
- The Snakemake pipeline files . . . 5
- Reading order . . . 7
- Project layout . . . 8
- 1. Configuration: `configs/parameters.py` . . . 8
- 2. Hi-C data preparation . . . 9
- 3. CTCF annotation . . . 10
- 4. The LEF dynamics engine . . . 10
- 5. Running simulations . . . 11
- 6. Post-simulation analysis . . . 13
- Legacy cluster scripts (`legacy/cluster/`) . . . 14
- Conventions you'll see everywhere . . . 15
- Common pitfalls . . . 15

CODE_GUIDE: What Every File in This Repo Actually Does

This is a plain-language walkthrough of the whole codebase. It is written for people who don't read Python and want to understand, end-to-end, what runs, what each script produces, and how the files feed into each other.

If you just want to run the pipeline, see `README.md`. Come back here when you need to know why a step exists, what an output file contains, or where a specific variable is set.

Quick-Start and Codebase Walkthrough

What this pipeline does

ChromSimPipe is a physics-based simulation framework for testing mechanistic hypotheses about 3D chromatin organisation. Given CTCF binding data and Hi-C maps, it simulates cohesin loop extrusion under different parameter conditions and compares the resulting contact maps to experimental Hi-C.

The included example (STRS project, Flores et al. 2026) asks: can we explain the stress-induced changes in Hi-C contact maps purely from what we can measure — changes in CTCF binding and cohesin dynamics — without invoking anything we can't observe?

The simulation models cohesin loop extrusion: cohesin rings load onto chromatin and extrude DNA into loops, stalling when they encounter CTCF sites oriented the right way. Where cohesins stall and how long they live determines the 3D contact map. By varying which CTCF sites are present and cohesin dynamics parameters, the simulation tests user-defined hypotheses.

Example conditions (shipped in **`config/ChromSimConfig.yaml`**):

Condition	Cohesin	CTCF sites	Tests
`cond1_baseline`	Baseline	Condition 1	Does the model reproduce condition 1 Hi-C?
`cond2_ctcf-only`	Baseline	Condition 2	Do condition 2 CTCF sites alone explain condition 2 Hi-C?
`cond2_weak-ctcf`	Baseline	Condition 2 (0.5× stall)	Does weaker stalling at remaining CTCF sites help?

Data flow

```
CTCF peaks (.narrowPeak)
    ↓  extract_ctcf_sites_hg38.py   (add strand orientation via FIMO)
Oriented CTCF BED files
    ↓  run_simulation.py + polychrom  (cohesin loop extrusion + 3D polymer)
Simulated contact maps
    ↓  run_analysis_all.py
Comparison figures  ↔  experimental Hi-C (.mcool, from .hic via hic2cool)
```

Setup: run this once

Edit `config/ChromSimConfig.yaml` (Hi-C paths, locus, conditions) and `ChromSimSamplesheet.txt` (CTCF peak paths) before running.

```
cd ~/projects/ChromSimPipe   # or wherever you cloned it
bash setup_data.sh
```

`setup_data.sh` reads all paths from `ChromSimConfig.yaml` and the samplesheet, then:

1. Step −1 — creates the two conda environments (`cohesin_sim` and `ctcf_extraction`) if they don't already exist.
2. Steps 1–3 — symlinks your `.hic` Hi-C files and converts them to `.cool` at 1 kb resolution (~10–15 min; run interactively: `srun --mem=32G --cpus-per-task=4 --pty bash`).
3. Step 4 — runs `extract_ctcf_sites_hg38.py` via `conda run -n ctcf_extraction`. Uses FIMO to find the JASPAR CTCF motif in each peak and assign strand orientation.
4. Step 6 — validates all BED files before any simulation runs.

After setup, pick a locus in `config/ChromSimConfig.yaml`:

```
locus:
  name: chr1_fig1   # chr1:64,210,000–66,170,000  (JAK1/AK4/LEPR)
  # other options: chr4_fig1, chr6_fig1, chr16_sox8
```

Then launch:

```
sbatch ChromSimPipe.sh
```

The central config: **`config/ChromSimConfig.yaml`**

This YAML file is the source of truth for everything. The `locus` section selects the genomic region; from that, `CHROM`, `REGION_START`, `REGION_END`, and `N_MONOMERS` (2,000 — one per kb) are derived automatically by `configs/parameters.py` at import time.

CTCF auto-loading. When any script imports `configs.parameters`, it immediately tries to open the oriented CTCF BED for each CTCF set and locus (e.g. `data/ctcf_beds/ctcf_oriented_hg38_condition`). It parses each line into `(monomer_position, orientation)` where +1 means a forward-strand motif and −1 means reverse. If the files don't exist yet, it falls back to placeholder dummy sites and prints a warning.

Cohesin parameters. Each condition has a parameter set, a dict with `lifetime` (mean steps before cohesin falls off), `separation` (spacing between cohesins), `ctcf_capture` (probability of stalling per CTCF encounter), and `ctcf_release` (escape probability per step while stalled). Values come from Gabriele et al. 2022 (mESC measurements used as the human baseline).

Tiling. Rather than simulating 2,000 monomers, the simulation runs 70,000 monomers arranged as 28 back-to-back copies of the locus (each tile = 2,000 monomers + 250 padding on each side). All 28 tiles are independent, so each GPU run produces ~28× the contact statistics at no extra physics cost. Contact maps are folded back down to 2,000×2,000 for comparison.

The CTCF extraction: **scripts/extract_ctcf_sites_hg38.py**

Your `.narrowPeak` file has peak coordinates but no strand information. Orientation matters because a + CTCF site blocks a cohesin arm moving leftward, while a − site blocks one moving rightward — so a convergent pair (+ on the left, − on the right) creates a stably anchored loop, while a divergent pair does not.

This script:

1. Takes each peak's summit from the CUT&Tag narrowPeak file.
2. Extracts the DNA sequence at each peak with `bedtools getfasta`.
3. Runs FIMO to scan for the JASPAR MA0139.1 CTCF motif.
4. Uses the motif's strand as the site orientation.
5. Writes a BED6 file where column 6 is + or −.

The 1D engine: **scripts/lef_dynamics.py**

Before any 3D physics, loop extrusion is modelled as a purely 1D stochastic process. `LEFSimulator` runs this:

- Each cohesin is a pair of arms that step away from each other along the chromatin fiber, one lattice site per simulation tick.
- An arm stalls when it hits a CTCF site with the blocking orientation, with probability `ctcf_capture`. It escapes with probability `ctcf_release` per step.
- Arms also stall when they collide with another cohesin (steric exclusion).
- Each cohesin lives for a geometrically distributed number of steps with mean `lifetime`, then unbinds and is replaced at a random position.

The 3D simulation: **scripts/run_simulation.py**

This is where polychrom (OpenMM on GPU) runs. For each block:

1. `LEFSimulator.step()` advances all cohesins one tick on the 1D lattice.
2. Cohesin arm positions feed into polychrom as harmonic spring bonds between the two bridged monomers.
3. OpenMM integrates the polymer under those bonds for 250 MD steps.
4. Every 10 blocks, all monomer coordinates are saved and pairs within 3.0 monomer-units of each other are recorded as contacts.

The analysis: **analysis/**

- **contact_maps.py** — accumulates saved conformations into a 2D contact frequency matrix using `scipy.spatial.KDTree` for fast distance lookups. This is the simulated analogue of a Hi-C contact map.

- **ps_curve.py** — computes P(s): contact probability as a function of genomic separation. The shape of this curve is the primary comparison metric against real Hi-C.
- **experimental_compare.py** — loads your `.mcool` file via `cooler` and extracts the same locus at 1 kb resolution for direct comparison.
- **msd_two_point.py** — tracks mean-squared displacement of specific monomer pairs over time (the in-silico analogue of live-cell imaging).

The cluster pipeline: **ChromSimPipe.sh** + Snakemake

The pipeline is a Snakemake DAG (`workflows/ChromSimPipe.snakefile`) launched by a single `sbatch ChromSimPipe.sh`. Snakemake submits each rule as its own SLURM job through `snakemake-executor-plugin-slurm`:

```
convert_hic (×2, CPU)
    → extract_ctcf (×2, CPU, FIMO)
        → validate_ctcf (×1, CPU)
            → simulate_shard (×36, GPU — 3 cond × 3 rep × 4 shards)
                → merge_shards (×9, CPU — 3 cond × 3 rep)
                    → analyze_all (×3, CPU — one per condition)
```

To dry-run (see all jobs without submitting): `bash ChromSimPipe.sh --dry-run`. If a run fails and leaves a lock: `bash unlock.sh`.

See The Snakemake pipeline files below for a full description of every new file.

The Snakemake pipeline files

These files were added to replace the legacy `cluster/submit_pipeline.sh` bash-script chain with a proper Snakemake DAG. Each downstream rule only runs after its inputs exist — no manual dependency bookkeeping required.

ChromSimPipe.sh — sbatch entry point

Submit with `sbatch ChromSimPipe.sh`. It does three things:

1. Sets up a local Python venv in `.snakemake_venv/` (first run only; uses `module load python/3.12.4`).
2. `pip installs` Snakemake 8.27.1 + `snakemake-executor-plugin-slurm`.
3. Calls `snakemake --profile profiles/slurm --configfile config/ChromSimConfig.yaml --jobs 100 --rerun-incomplete --latency-wait 500`.

The venv is reused on subsequent runs. `bash ChromSimPipe.sh --dry-run` prints every SLURM command without submitting.

`unlock.sh` — release lock after failure

Snakemake writes a `.snakemake/locks/` directory while running. If the job dies without cleanup, the lock persists and the next run refuses to start. `bash unlock.sh` runs `snakemake --unlock` against the workflow to clear it.

`workflows/ChromSimPipe.snakefile` — the DAG

Six rules, in order:

Rule	SLURM resources	What it does
`convert_hic`	4 CPU, 32 GB, 2 h	`hic2cool convert .hic → .cool` at 1 kb (single-resolution)
`extract_ctcf`	2 CPU, 16 GB, 1 h	`extract_ctcf_sites_hg38.py` via `ctcf_extraction` env
`validate_ctcf`	1 CPU, 4 GB, 15 min	`validate_ctcf.py` — sanity-checks all BED files
`simulate_shard`	1 GPU, 4 CPU, 64 GB, 12 h	`run_simulation_shard.py` — 36 jobs with the example config (3 × 3 × 4)
`merge_shards`	4 CPU, 64 GB, 2 h	`merge_shards.py` — concatenates shard HDF5 streams
`analyze_all`	4 CPU, 64 GB, 6 h	`run_analysis_all.py` — contact maps, P(s), MSD, figures

Wildcard `{condition}` matches the 5 condition names from `SIMULATION_CONDITIONS` in `configs/parameters.py`. Wildcards `{rep}` and `{shard}` expand over `range(n_replicates)` and `range(n_shards)` from `config/SimConfig.yaml`.

Locus, conditions, and cohesin parameters all come from `configs/parameters.py` at parse time — there is no duplication.

Snakemake tracks completion via sentinel `.done` files in `results/.sentinels/`. Actual simulation data lives in `results/polychrom_3d/` with the flat naming convention the scripts already expect.

`profiles/slurm/config.yaml` — SLURM executor defaults

Tells `snakemake-executor-plugin-slurm` which account and partition to use for all rules by default. Per-rule overrides (GPU partition, GPU count, memory, walltime) are set in each rule's `resources:` block in the Snakefile.

Key defaults:

- `slurm_account: rc_dphansti_pi`
- `slurm_partition: general` (GPU rules override to `gpu`)
- `runtime: 4320` (72 h fallback for long-running jobs)
- Log files: `logs_slurm/{rule}.{wildcards}.{jobid}.out/err`

`config/ChromSimConfig.yaml` — path configuration

Edit once before the first run. Key fields:

Field	What to change
`hic`	One `<hic_type>: /abs/path.hic` entry per Hi-C map (e.g. `condition1`, `condition2`); each condition's `compare_hic` names one
`genome`	Path to hg38 FASTA (default: Longleaf shared reference)
`samplesheet`	Path to `SimSamplesheet.txt`
`n_replicates`	Independent simulation replicates (default: 3)
`n_shards`	GPU shards per replicate (default: 4, gives 4× frames per GPU run)

All other parameters (locus, cohesin params, tiling) are read from `configs/parameters.py` and do NOT need to be duplicated here.

`ChromSimSamplesheet.txt` — CTCF input file registry

Tab-separated, two columns: `CTCF_Type` and `CTCF_Peaks_Path`.

```
CTCF_Type   CTCF_Peaks_Path
condition1  /path/to/condition1_CTCF_peaks.narrowPeak
condition2  /path/to/condition2_CTCF_peaks.narrowPeak
```

The `extract_ctcf` rule reads this file via `pandas.read_table()` at runtime. If your data moves, update the paths here (not in the Snakefile).

Reading order

Everything here is organised by pipeline stage, top-down:

1. The Snakemake pipeline files — start here
2. Project layout — what each folder is for
3. Configuration — the single file that controls every simulation
4. Hi-C data preparation — turning experimental data into comparison matrices
5. CTCF annotation — turning CUT&Tag peaks into oriented barrier lists
6. The LEF dynamics engine — the loop-extrusion simulator
7. Running simulations — the shard, merge, and single-run entry points
8. Post-simulation analysis — contact maps, P(s), APA, MSD, calibration
9. Legacy cluster scripts — old sbatch wrappers (superseded by ChromSimPipe.sh)
10. Conventions you'll see everywhere — monomers, tiling, output directory names

11. Common pitfalls — what to check when something looks wrong

Every script section follows the same format:

- File — path in the repo
- Purpose — one-sentence summary
- Inputs — command-line flags and files read
- Outputs — files written, with exact paths
- How to run it — example command
- What it does, step by step — plain-language walkthrough

Project layout

```
ChromSimPipe/
├── config/           — ChromSimConfig.yaml (edit before first run)
├── configs/       — parameters.py (reads YAML; single source of truth for all params)
├── data/          — experimental inputs (Hi-C .cool files, oriented CTCF beds)
├── envs/             — conda environment specs (.yml files)
├── scripts/          — Python simulation, analysis, and utility scripts
├── analysis/       — Python modules imported by scripts/run_analysis_all.py
├── workflows/        — Snakemake DAG (ChromSimPipe.snakefile)
├── profiles/slurm/   — SLURM executor configuration for Snakemake
├── legacy/        — unmaintained code from the lab's mESC/neuron project (see legacy/READM
├── tests/            — test suite
├── results/          — simulation and analysis outputs (git-ignored)
└── logs/             — top-level Snakemake job stdout/stderr
                        (per-rule job logs live in .snakemake/slurm_logs/)
```

1. Configuration: **`configs/parameters.py`**

Purpose. This file is the runtime source of truth for every simulation parameter. It reads `config/ChromSimConfig.yaml` at import time and re-exports a fixed set of variables so that all downstream scripts stay unchanged when config values change.

Key exported names:

- Locus (`CHROM`, `REGION_START`, `REGION_END`, `RESOLUTION`, `N_MONOMERS`, `ACTIVE_LOCUS`). Derived from the `locus:` block in `ChromSimConfig.yaml`. For the included STRS example: hg38, ~2 Mb windows in chr1/chr4/chr6/chr16. At 1 kb/monomer, each window → ~2,000 monomers.
- Polymer physics (`POLYMER` dict). Bond lengths, bending stiffness (`angle_force`), confinement density, integration tolerance, GPU platform. Rarely changed between projects.
- LEF ↔ polymer bonds (`SMC_BOND`). How stiff the cohesin “ring” is in 3D — how strongly it pulls the two loop flanks together.

- CTCF tracks (`CTCF_TYPES`, `CTCF_BEDS`). One oriented BED per `ctcf_type` used by a condition, under `data/ctcf_beds/`. At import time, `load_ctcf_from_bed()` reads each file into `CTCF_SITES_BY_TYPE[type]` — monomer indices with +1 / −1 orientation labels. Type names come from the config, so any number of CTCF conditions is supported.
- Cohesin parameter sets (`ALL_PARAM_SETS`). One dict per condition hypothesis, each with `lifetime`, `separation`, `ctcf_capture`, and `ctcf_release`. Values from Gabriele et al. 2022.
- Condition list (`SIMULATION_CONDITIONS`). The rows the Snakemake pipeline iterates over. Each entry pairs a cohesin parameter set with a CTCF track and has a `name` (e.g. `cond1_baseline`).
- Tiling (`TILING`). The 2 Mb locus is copied 28× into one long polymer for 28× better contact statistics at no extra physics cost. `TILING["chrom_size"]` is the full polymer length (~70,000 monomers).
- Run parameters (`SIM_RUN`). `total_blocks`, `md_steps`, `save_every`, `warmup_blocks`.
- Calibration (`CALIBRATION`). Whether to map simulation units → physical units via Hi-C P(s) alignment, MSD, or skip.

Helper functions you'll see called elsewhere:

- `get_condition(name)` — look up one entry in `SIMULATION_CONDITIONS`.
- `list_conditions()` — list all condition names.
- `get_ctcf_arrays(ctcf_type)` — returns `(positions, orientations)` for the untiled 2 Mb locus.
- `get_tiled_ctcf_arrays(ctcf_type)` — same but repeated across the tiled polymer (what the simulations actually use).
- `load_ctcf_from_bed(path)` — read a BED6 file and return `(positions, orientations)` as monomer-index arrays.

2. Hi-C data preparation

In ChromSimPipe, Hi-C conversion is handled automatically by the `convert_hic` Snakemake rule: it calls `hic2cool convert` on each `.hic` file listed in `config/ChromSimConfig.yaml` to produce a single-resolution `.cool` at the configured resolution (default 1 kb). No manual steps required — just point the config at your `.hic` files.

The Bonev 2017 mESC/cortical-neuron download scripts and the Sox2 quick-look plotter from the original lab project now live in `legacy/` (see `legacy/README.md`); they are not part of the workflow.

3. CTCF annotation

`scripts/extract_ctcf_sites_hg38.py`

Purpose. Given a CTCF ChIP-seq or CUT&Tag peak BED and a genome FASTA, run FIMO against the JASPAR MA0139.1 CTCF motif to assign strand orientation to each peak, and write an oriented BED6 file.

The `extract_ctcf` Snakemake rule calls this automatically for every sample in `ChromSimSamplesheet.txt`. To run manually:

```
conda run -p ~/.local/share/mamba/envs/ctcf_extraction \
    python scripts/extract_ctcf_sites_hg38.py \
    --source bed \
    --bed /path/to/peaks.narrowPeak \
    --region chr1:64210000-66170000 \
    --genome /path/to/hg38.fa \
    --output data/ctcf_beds/ctcf_oriented_hg38_condition1_chr1_64210000_66170000.bed
```

Output: BED6 — one row per peak with a usable FIMO hit; column 6 (+/−) is the motif strand.

`scripts/validate_ctcf.py`

Purpose. Pre-flight sanity check before any simulation. Loads the oriented CTCF BEDs, converts genomic coordinates to monomer indices, and prints a report. Called by the `validate_ctcf` Snakemake rule; also runnable standalone:

```
conda run -p ~/.local/share/mamba/envs/cohesin_sim python scripts/validate_ctcf.py
```

Checks: BED parses cleanly; all sites are inside the simulation window; coordinate → monomer-index mapping is monotone; prints counts per condition and sites gained / lost / shared. Exits non-zero on failure.

`scripts/export_conserved_ctcf_bed.py`

Purpose. Intersect two oriented CTCF BEDs, keep only sites shared in both (same position and orientation), and write a conserved-sites BED. Useful for testing "CTCF binding didn't change, only cohesin did" scenarios.

4. The LEF dynamics engine

`scripts/lef_dynamics.py`

Purpose. A lightweight 1D Python simulator for loop-extruding factors (cohesins). No 3D physics — just: where are the cohesins on the lattice, where are they stalled, where do they jump when they unbind.

Class: **`LEFSimulator`**. Constructor arguments:

- `N` — lattice length in monomers.
- `n_lefs` — how many cohesins to place.
- `lifetime` — mean number of steps a cohesin stays bound before unbinding (`unbind probability per step = 1/lifetime`).
- `ctcf_positions`, `ctcf_orientations` — from `get_ctcf_arrays`.
- `ctcf_capture` — probability of stalling when a cohesin arm moves onto a CTCF site with the blocking orientation.
- `ctcf_release` — probability per step that a stalled cohesin lets go.
- `rng_seed` — for reproducibility.

What each **`step()`** call does.

1. Rebuild the occupancy grid from every cohesin's left / right arm positions (so collisions are up-to-date if arms moved last step).
2. For each cohesin (in a random permutation, so no directional bias):
    - Age by one step.
    - With probability `1/lifetime`, unbind → clear occupancy, call `_place_lef()` to re-seed at a random free pair of positions.
    - Left arm (moves leftward): if the next position is free and not a blocking CTCF site (or we rolled `> ctcf_capture`), move there. If it is blocking and we rolled `< ctcf_capture`, flag as stalled.
    - Right arm (moves rightward): mirror logic.
    - If already stalled on CTCF, roll `< ctcf_release` to escape.

Public methods you'll see used.

- `get_bonds()` — list of `(left, right)` tuples of current cohesin-bridged monomer pairs. Fed to polychrom as harmonic SMC bonds.
- `get_bond_arrays()` — same but as two parallel NumPy arrays.
- `get_loop_sizes()` — current loop lengths (in monomers).
- `run(n_steps)` — convenience: step `n_steps` times, return the list of per-step bond lists.

Output variables you should know. After a `step()`, these arrays reflect the current state: `left_pos[i]`, `right_pos[i]`, `ages[i]`, `left_stalled[i]`, `right_stalled[i]`, `occupied[pos]`.

5. Running simulations

5a. **`scripts/run_simulation_shard.py`** — the Snakemake entry point

This is what every `simulate_shard` SLURM job calls (36 jobs with the example config: 3 conditions × 3 replicates × 4 shards).

Arguments:

Flag	Meaning
`--condition NAME`	One of the names in `SIMULATION_CONDITIONS`. Sets cohesin params + CTCF track.
`--replicate N`	Integer replicate index (drives RNG seed).
`--shard-index I`	Which shard of this replicate (0…`n_shards`−1).
`--n-shards K`	Total shards per replicate (default 4).
`--gpu I`	CUDA device index.
`--output DIR`	Output base directory (default `results/`).
`--shard-index-offset M`	Add `M` to shard index — use to extend existing runs without overwriting.

Shard output goes to `<output>/<condition>_rep<r>_shard<i>/`.

What it does:

1. Look up the condition → resolve to `(params_dict, ctcf_type)`.
2. Build the tiled polymer (`N = TILING["chrom_size"]`), fetch tiled CTCF arrays.
3. Warm up `LEFSimulator` for `warmup_blocks` steps (throw away).
4. Pre-compute all LEF bond states for every block in advance — the 1D dynamics are independent of 3D physics, so doing this up front lets the polychrom loop replay bonds from a list.
5. Register all unique bond pairs as `harmonic_bonds` at `k=0`; flip `k` each block (~10× faster than adding/removing bonds).
6. `sim.local_energy_minimization()` relaxes the starting configuration.
7. Run `total_blocks / n_shards` blocks; `HDF5Reporter` saves conformations every `save_every` blocks.

5b. **`scripts/merge_shards.py`**

Concatenates per-shard HDF5 streams into `merged_<condition>_rep<N>/` with the same layout the analysis expects. Writes `merge_metadata.json` recording which shard IDs contributed.

`--condition, --replicate, --n-shards, --results-dir`

5c. **`scripts/run_simulation.py`** — single-run / local testing

One-shot simulation without sharding. Useful for local testing or LEF-only sanity checks:

```
conda run -p ~/.local/share/mamba/envs/cohesin_sim \
    python scripts/run_simulation.py \
    --condition cond1_baseline --replicate 0 --gpu 0 --output results/
```

`--engine lef_only` skips the 3D polymer entirely and writes only a 2D contact map from 1D LEF statistics.

6. Post-simulation analysis

6a. The orchestrator: **`scripts/run_analysis_all.py`**

One command that walks every `merged_<condition>_rep<N>` directory, runs the full analysis suite, and writes per-condition outputs. Called by the `analyze_all` Snakemake rule.

How to run directly (e.g. for one condition):

```
conda run -p ~/.local/share/mamba/envs/cohesin_sim \
    python scripts/run_analysis_all.py \
    --results-dir results/polychrom_3d \
    --output-dir results/analysis/cond1_baseline \
    --condition cond1_baseline \
    --hic-dir data/mcool \
    --n-jobs 4 --skip-existing --no-msd
```

Selected flags:

Flag	What it does
`--results-dir DIR`	Where to find merged simulation directories.
`--output-dir DIR`	Where to write outputs.
`--condition NAME`	Analyse one condition only.
`--n-jobs N`	Parallel workers for contact detection.
`--skip-existing`	Skip dirs that already have `sim_contact_map.npy`.
`--no-msd`	Skip MSD computation (disabled by default in the Snakemake rule).
`--hic-dir DIR`	Where to find `<compare_hic>.cool` for each condition (falls back to `data/mcool/`).
`--mcool TYPE=PATH`	Override the `.cool` for one Hi-C type (repeatable).
`--ctcf-bed TYPE=PATH`	Override the oriented CTCF BED for one ctcf_type (repeatable).

Per-replicate outputs (in `<sim_dir>/analysis/`):

File	What it is
`sim_contact_map.npy`	2D contact matrix from 3D conformations
`sim_ps_curve.npz`	Contact probability vs genomic distance
`sim_insulation.npy`	Insulation score along the locus
`apa_convergent.png`	Aggregate Peak Analysis at convergent CTCF pairs
`sim_vs_exp_map.png`	Side-by-side sim vs experimental Hi-C heatmap
`comparison_metrics.json`	Pearson + stratum-adjusted correlations with experimental Hi-C
`msd_<label>.npz`	Two-point MSD curves (if `--no-msd` not set)
`calibration.json`	Physical-unit mapping: nm/monomer, sec/frame

6b. Analysis modules (**`analysis/`**)

Module	Purpose
`contact_maps.py`	Contact matrix from conformations via `scipy.spatial.cKDTree`; also P(s), insulation, map-vs-map correlation
`ps_curve.py`	Power-law fitting of P(s)
`absolute_quant.py`	Aggregate Peak Analysis (APA) at convergent CTCF pairs
`ctcf_plotting.py`	Contact-map heatmaps with CTCF arrow strips
`experimental_compare.py`	Sim vs experimental Hi-C comparison (`.cool` and `.npy`)
`msd_two_point.py`	Mean-squared displacement of monomer pairs; sub-diffusive exponent fit
`polymer_dynamics.py`	Radius of gyration, loop fractions, dwell-time distributions
`calibration.py`	Map simulation units → physical units (nm/monomer, sec/frame)
`batch_analysis.py`	Older single-pass analysis over all result dirs (simpler than the orchestrator)

6c. **`scripts/summarize_analysis.py`**

Collects scalar metrics from all conditions into one CSV/XLSX table:

```
python scripts/summarize_analysis.py \
    --analysis-dir results/analysis \
    --reference-condition cond1_baseline
```

6d. **`scripts/check_results.py`**

Lists every directory under `results/polychrom_3d`, its frame count, and whether analysis outputs are present. Quick inventory after a SLURM batch.

Legacy cluster scripts (**`legacy/cluster/`**)

The pre-Snakemake SLURM scripts were written for the lab's original cluster (mambaforge under `ric.tiget`, `cuda` / `workq` partitions) and the mESC/neuron study, so they have been moved to `legacy/cluster/` and are not maintained. Use **`sbatch ChromSimPipe.sh`**, which handles the full DAG on Longleaf.

afterok vs **afterany**

- `--dependency=afterok:<JID>` — fire only if upstream finished with exit 0. Use between pipeline stages where failure upstream makes the next stage meaningless.
- `--dependency=afterany:<JID>` — fire regardless of how upstream ended. Use for resume-chain links meant to recover from wall-time TIMEOUTs. `submit_analysis_resume_chain.sh` uses this pattern.

Conventions you'll see everywhere

- Monomer = 1 kb. One bead ↔ 1,000 bp. Monomer index `i` covers `REGION_START + i * RESOLUTION` to `REGION_START + (i+1) * RESOLUTION`.
- Tiled chromosome. Simulations run on ~70,000 monomers = 28 copies of the 2 Mb locus with padding. Contact maps fold back to `N_MONOMERS` (~2,000) for comparison. This gives ~28× more contact statistics at no extra physics cost.
- Output directory names. Shards: `<condition>_rep<N>_shard<S>/`. Merged: `merged_<condition>_rep<N>/`. The `_ctcf-<type>` in the condition name (e.g. `cond1_baseline`) is the key separator between runs with the same cohesin params but different CTCF tracks.
- RNG seed. `42 + replicate * 10000 + shard * 1000`. Always derived from indices so reruns are reproducible.
- Every run writes **`params.json`** with the full parameter dict + CTCF type + replicate. If a result looks wrong, check `params.json` first.
- Sentinel files. Snakemake tracks completion via `.done` files in `results/.sentinels/`. Don't touch these manually; delete the relevant `.done` file if you need to re-run a step.

Common pitfalls

CUDA architecture error. `openmm.OpenMMException: nvrtc: error: invalid value for --gpu-architecture (-arch)` means the OpenMM/CUDA in `cohesin_sim` was compiled for a different GPU microarchitecture than the allocated node. Usually resolves by resubmitting (different node) or reinstalling OpenMM with a CUDA toolkit that matches the target GPU.

Snakemake lock error after a failed run. Run `bash unlock.sh`, then resubmit.

`ModuleNotFoundError: configs` or **`analysis`**. Every script in `scripts/` inserts the repo root onto `sys.path`. Run from the repo root, or set `PYTHONPATH=/path/to/ChromSimPipe`.

`KeyError: 'CTCF_Type'` in samplesheet. `ChromSimSamplesheet.txt` must be tab-separated with headers exactly `CTCF_Type` and `CTCF_Peaks_Path`.

`hic2cool` fails. Check `config/ChromSimConfig.yaml`: every entry under `hic:` must be an absolute path to an existing `.hic` file.

Analysis fails with "No results found". Check that `results/polychrom_3d/merged_<condition>_rep*/` directories exist. If shards completed but the merged directory is missing, run `merge_shards.py` manually or re-trigger the Snakemake `merge_shards` rule.

Per-job SLURM logs location. Per-rule job logs go to `.snakemake/slurm_logs/rule_<name>/<wildcards>/<` not to `logs/`. The `logs/` directory only contains the top-level Snakemake process output (e.g. `logs/snakemake_<jobid>.err`).